

Auto A11y REST API Guide

A practical guide to the Auto A11y REST API: authentication, conventions, and the workflows that cover most integrations. For the complete, authoritative contract of every endpoint (request/response schemas, all error codes), use the OpenAPI 3.1 specification:

- **In this repo:** [docs/api/openapi.yaml](#)
- **From a running instance:** GET `/api/v1/openapi.json` or GET `/api/v1/openapi.yaml`
- **Regenerate after route changes:** `python scripts/generate_openapi.py --check-exits` non-zero if the committed spec is stale)

Contents

1. [Basics](#)
 2. [Authentication](#)
 3. [Conventions](#)
 4. [Core workflow: from project to report](#)
 5. [The async job model](#)
 6. [Endpoint reference by area](#)
 7. [Errors](#)
-

Basics

Item	Value
Base URL	<code>http://127.0.0.1:5001/api/v1</code> (default port is 5001)
Content type	<code>application/json</code> for request and response bodies
Auth	Bearer token (Authorization header) or session cookie
Errors	RFC 7807 <code>application/problem+json</code>
Rate limit	60 requests/minute by default (<code>RATELIMIT_DEFAULT</code>)
IDs	MongoDB ObjectId strings (24 hex characters)

A quick liveness check needs no authentication:

```
curl http://127.0.0.1:5001/api/v1/health
```

Authentication

The API uses **Bearer tokens**. Exchange credentials for a token once, then send it on every request:

```
curl -X POST http://127.0.0.1:5001/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "you@example.org", "password": "...", "description": "CI pipeline"}'
```

Response (200):

```
{
  "token": "raw-token-value",
  "token_record": { "...": "metadata about the token" },
  "user": { "...": "your user record" }
}
```

The raw token is returned exactly once. The server stores only its SHA-256 hash, so persist the value client-side immediately. The optional `description` labels the token so it can be identified (and revoked) later on the tokens page.

Use it in the Authorization header:

```
curl http://127.0.0.1:5001/api/v1/projects \
  -H "Authorization: Bearer $TOKEN"
```

Notes:

- `POST /auth/logout` revokes the token used to make the call.
- Tokens can expire (`expires_at`) and can be revoked by an admin; expect `401` and re-login when that happens.
- Login failures always return an opaque `401 invalid credentials` — the API deliberately does not distinguish unknown email / wrong password / locked account (defends against enumeration).
- Browser frontends may authenticate with the Flask session cookie instead (`sessionAuth` in the spec); pass `"session": true` to `/auth/login` to set it. Script and CI clients should stick to Bearer tokens.
- SSO deployments: `GET /auth/sso/{provider}/url` → redirect the user → `GET /auth/sso/{provider}/callback` mints the Bearer token.

Conventions

Verbs. `GET` reads, `POST` creates (or queues work), `PUT` replaces, `PATCH` partially updates, `DELETE` deletes. Resources with both `PUT` and `PATCH` accept either a full replacement or a sparse update.

Asynchronous work returns 202. Anything that drives a browser — discovery crawls, test runs, report generation, PDF audits — is queued as a background job. The `202` response carries a `job_id` (or `run id`); poll the corresponding status endpoint. See [the async job model](#).

Errors are RFC 7807 problem documents. See [Errors](#).

List envelopes. Collection endpoints return a wrapper object rather than a bare array — e.g. GET /projects returns {"projects": [...], "pagination": {...}}. Check the spec for each endpoint's envelope shape.

Core workflow: from project to report

The typical integration touches five resources in order: **project** → **website** → **discovery** → **test runs** → **report**.

1. Create a project

```
curl -X POST $BASE/projects \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"name": "Corporate site audit", "description": "Q3 audit"}'
```

Only name is required. The response includes the project id.

2. Add a website

```
curl -X POST $BASE/projects/$PROJECT_ID/websites \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"url": "https://example.org", "name": "Main site"}'
```

Only url is required; `scraping_config` can tune crawl depth and limits.

3. Discover pages

Queue a crawl; it returns 202 with a run id:

```
curl -X POST $BASE/websites/$WEBSITE_ID/discoveries \
  -H "Authorization: Bearer $TOKEN"
```

Poll until it completes:

```
curl $BASE/websites/$WEBSITE_ID/discoveries/latest \
  -H "Authorization: Bearer $TOKEN"
```

Then list what was found:

```
curl $BASE/websites/$WEBSITE_ID/pages -H "Authorization: Bearer $TOKEN"
```

Pages can also be created directly (POST /websites/{id}/pages) if you already know the URLs and want to skip crawling.

4. Run accessibility tests

Test one page, a whole website, or a whole project — each queues background work and returns 202:

```
# one page
curl -X POST $BASE/pages/$PAGE_ID/test-runs -H "Authorization: Bearer $TOKEN"

# every testable page in a website
curl -X POST $BASE/websites/$WEBSITE_ID/test-runs -H "Authorization: Bearer $TOKEN"

# every website in a project
curl -X POST $BASE/projects/$PROJECT_ID/test-runs -H "Authorization: Bearer $TOKEN"
```

The website-scope response looks like:

```
{
  "job_id": "test_ab12cd34",
  "status": "queued",
  "pages_queued": 42,
  "total_tests": 42,
  "user_count": 1,
  "website_id": "...",
  "message": "..."
}
```

Poll GET /websites/{id}/test-runs/latest (or GET /jobs/{job_id}) until the run finishes. Cancel with the matching .../cancel endpoint.

5. Read the results

```
# latest issue buckets for a page: violations, warnings, info,
# discovery, ai_findings
curl $BASE/pages/$PAGE_ID/violations -H "Authorization: Bearer $TOKEN"

# full result history for a page
curl $BASE/pages/$PAGE_ID/test-results -H "Authorization: Bearer $TOKEN"

# one specific result
curl $BASE/test-results/$RESULT_ID -H "Authorization: Bearer $TOKEN"
```

Pages tested in multiple states (e.g. with and without a cookie banner) expose per-state results via GET /pages/{id}/test-states and GET /test-results/{id}/states.

6. Generate and download a report

Queue a report at the scope you want. type selects the report kind — accessibility (default), discovery, recordings, or deduplicated — and format the output format (defaults to xlsx; the accessibility type also supports html, csv, json, and pdf):

```
curl -X POST $BASE/projects/$PROJECT_ID/reports \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type:
```

```
application/json" \
-d '{"type": "accessibility", "format": "html"}'
```

The generic entry point `POST /reports` accepts `{project_id | website_id | page_id, type, format, include_ai}` instead. The 202 response carries a `job_id`; poll `GET /jobs/{job_id}` until status is completed, then download the file using that same job id:

```
curl -L -o report.html $BASE/reports/$JOB_ID/file \
-H "Authorization: Bearer $TOKEN"
```

`GET /projects/{id}/report-summary` aggregates a project's completed report jobs (useful for a “reports” listing UI).

The async job model

Every queued operation is tracked as a job:

Endpoint	Purpose
<code>GET /jobs/{job_id}</code>	Status of one job: status, progress, result, error, timestamps
<code>GET /jobs/active</code>	All currently running/pending jobs
<code>POST /jobs/{job_id}/cancel</code>	Request cancellation (workers observe the flag at their next progress tick)
<code>POST /jobs/{job_id}/restart</code>	Restart a failed report-generation job
<code>GET /jobs/stats</code>	Counts by type/status over the past 24 h
<code>POST /jobs/clear-stale</code>	Housekeeping: clear jobs older than 24 h

Poll every few seconds while a job runs. Terminal statuses are `completed`, `failed`, and `cancelled`; `result` (on success) or `error` (on failure) carries the outcome.

Endpoint reference by area

Summaries only — request/response schemas live in the OpenAPI spec. Path parameters are elided where obvious.

Projects, websites, pages

The core resource hierarchy: projects contain websites, websites contain pages. All three support standard CRUD (GET/POST on collections; GET/PUT/PATCH/DELETE on items). Notables beyond CRUD:

- `GET /pages/{id}/matrix`, `PUT /pages/{id}/matrix` — the page's test-state matrix (which states × breakpoints to test)
- `GET /websites/{id}/documents` — document references found on a website
- `DELETE /websites/{id}/test-results` — clear a website's results

- GET /projects/{id}/discovered-pages, POST ... and GET|PUT|PATCH|DELETE /discovered-pages/{id} — manually-tracked page inventory distinct from crawled pages

Test runs & discoveries

Queueing and monitoring browser work (see workflow above). Also:

- GET /test-runs — list runs; POST /test-runs (top-level, takes page_id), POST /test-runs/batch for several pages at once
- GET /discoveries/{run_id}, POST /discoveries/{run_id}/cancel

Test results & analytics

- GET /pages/{id}/test-results, /test-sessions, /test-states, GET /test-results/{id}, GET /test-results/{id}/states
- Trends: GET /test-runs/trends, .../compare, .../detailed, .../progress
- GET /testing/stats — aggregate stats, optionally scoped
- GET|PUT /testing/config — runtime testing configuration

Reports

Queue (POST /reports, or scoped under project/website/page), poll the job, download (GET /reports/{id}/file), delete, restart failed jobs, GET /projects/{id}/report-summary.

PDFs

Upload or fetch a PDF (POST /projects/{id}/pdfs), start audits (POST /pdf-documents/{id}/audits), poll (GET .../audits/latest), cancel, stream the file/page images, and export a self-contained audit report (GET /pdf-documents/{id}/export, Markdown or HTML).

Recordings (lived-experience testing)

Upload Dictaphone JSON recordings (POST /recordings), CRUD on recordings and their issues, and supplementary content merge (GET|PATCH /recordings/{id}/content).

Scheduling

CRUD for scheduled tests under GET|POST /websites/{id}/scheduled-tests and /scheduled-tests/{id}, plus GET .../preview (next run times) and POST .../runs (trigger immediately).

Test users & setup scripts

Machinery for testing behind logins and complex page states:

- Test users (login credentials the test runner uses) at project and website scope: .../test-users collections, /project-test-users/{id} and

/website-test-users/{id} items, POST .../test-login (verify the login automation works), DELETE .../session-cache

- Setup scripts (actions to run before testing) at website and page scope: .../scripts collections, /scripts/{id} items, POST /scripts/{id}/test-runs (execute synchronously)

Access control & accounts

- GET|PATCH /auth/me, GET /users/me — the authenticated user
- /users — superadmin user administration (list, create, patch, delete, unlock)
- /groups — permission groups CRUD
- /projects/{id}/members — project membership and group assignments
- /projects/{id}/supervisors, /projects/{id}/testers — test supervisors and lived-experience testers
- /share-tokens (+ project/website-scoped creation) — revocable read-only share links

Integration & platform

- Drupal sync under /drupal/... — import pages/issues from Drupal, upload results, per-project sync status
- Fixtures: GET /fixture-tests/status, GET /fixture-tests/check/{code} — which accessibility checks are validated and production-enabled
- Issue catalog: GET /issues/{code}, documentation-status endpoints
- Admin settings under /admin/settings
- Health: GET /health, GET /health/pdf

Errors

Failures return an RFC 7807 problem document with Content-Type: application/problem+json:

```
{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404,
  "detail": "Project 507f1f77bcf86cd799439011 not found",
  "instance": "/api/v1/projects/507f1f77bcf86cd799439011",
  "errors": null
}
```

errors carries field-level validation details on 400 responses.

Status	Meaning
400	Malformed body or failed validation — see errors
401	Missing, expired, or revoked token; failed login
403	Authenticated but not authorized for

Status	Meaning
	this resource/scope
404	Resource does not exist (or is outside your visibility)
409	Conflict — e.g. cancelling a job that already finished
429	Rate limit exceeded — back off and retry
500	Server error — check the application log

See also: [API_DESIGN.md](#) (original design), [REST_API_ROADMAP.md](#) (implementation roadmap), and the generated [openapi.yaml](#) for the full contract.